



GH30x SDK 移植指南

版本：3.6

版权所有 © 2022 深圳市汇顶科技股份有限公司。保留一切权利。

非经本公司书面许可，任何单位和个人不得对本手册内的任何部分擅自摘抄、复制、修改、翻译、传播，或将其全部或部分用于商业用途。

商标声明

GOODIX 和其他汇顶商标均为深圳市汇顶科技股份有限公司的商标。本文档提及的其他所有商标或注册商标，由各自的所有人持有。

免责声明

本文档中所述的器件应用信息及其他类似内容仅为您提供便利，它们可能由更新之信息所替代。确保应用符合技术规范，是您自身应负的责任。

深圳市汇顶科技股份有限公司（以下简称“GOODIX”）对这些信息不作任何明示或暗示、书面或口头、法定或其他形式的声明或担保，包括但不限于针对其使用情况、质量、性能、适销性或特定用途的适用性的声明或担保。GOODIX 对因这些信息及使用这些信息而引起的后果不承担任何责任。

未经 GOODIX 书面批准，不得将 GOODIX 的产品用作生命维持系统中的关键组件。在 GOODIX 知识产权保护下，不得暗中或以其他方式转让任何许可证。

深圳市汇顶科技股份有限公司

总部地址：深圳市福田区腾飞工业大厦 B 座 2 层、13 层

电话：+86-755-33338828 传真：+86-755-33338099

网址：www.goodix.com

前言

编写目的

本文档主要介绍如何使用 GH30x SDK 文件，完成 GH30x 在对应平台的移植，以帮助开发者快速实现 GH30x 心率（HR）、血氧饱和度（SpO2）、心率变异性（HRV）和佩戴检测（NADT）等功能。

读者对象

本文适用于以下读者：

- GH30x 用户
- 软件开发工程师
- FAE 工程师

修订记录

版本	日期	修订内容
1.0	2019-07-03	首次发布
2.0	2020-07-20	根据 example 重构文档
3.0	2021-04-29	重构文档，增加驱动 API 说明
3.1	2021-05-14	修改 MCU 本地测试部分，为 TestLib V1.1.0 后版本
3.2	2021-09-03	修改增加部分驱动库 API 说明
3.3	2021-09-23	修改部分驱动库 API 说明，增加平滑策略说明
3.4	2022-03-21	修正描述，删除多余描述，整体格式修改
3.5	2020-07-14	新增部分内容，更新部分 API 说明
3.6	2023-06-04	V69 预发布版本相关更新

目录

前言	I
目录	II
1 简介	1
1.1 SDK 架构	1
1.2 SDK 应用流程	1
1.3 驱动和算法调用层文件	3
1.3.1 gh30x_example_config.h	4
1.3.2 gh30x_example.h	5
1.3.3 gh30x_example_port.c	5
1.3.4 gh30x_example_reg_array.c	7
1.3.5 gh30x_example_hook.c	8
1.3.6 gh30x_demo_algo_config.h	8
1.3.7 gh30x_demo_algo_hook.c	9
2 例程移植说明	9
2.1 基本功能移植	9
2.1.1 移植 SDK 到工程	9
2.1.2 调通通信功能	10
2.1.3 修改配置文件	11
2.1.4 GH30X 的初始化函数的调用	11
2.1.5 GH30X 的功能开启和关闭	11
2.1.6 GH30X 中断处理函数的调用	12
2.1.7 蓝牙透传功能移植	错误!未定义书签。
2.1.8 验证测试	12
2.2 应用方案介绍	12
2.2.1 应用框图	12
2.2.2 佩戴模块	13
2.2.3 关于算法并行的说明	14
2.3 应用调试	14
2.3.1 G-sensor 调试	14
2.3.2 蓝牙透传调试	22
2.3.3 量产测试调试	26
3 SDK API 说明	30
3.1 驱动层 API	30

3.2 算法调用层 API	33
3.2.1 函数描述	33
3.2.2 算法结果数组描述	33
4 常见问题及注意事项	36
5 附录	38
5.1 心率场景定义	38

1 简介

GH30x 驱动库为 GH30x 系列芯片（如 GH300、GH301、GH3011、GH3018）的应用开发提供驱动支持，包含丰富的应用 API、配置文件、硬件驱动接口以及 lib 文件等，可帮助应用开发者快速集成心率、血氧及佩戴检测功能，缩短产品开发周期。

1.1 SDK 架构

如图 1-1 所示，GH30x SDK 的应用软件架构如下：

- 驱动层：包含 GH30x 芯片的功能驱动程序及协议处理程序，并提供丰富的 API 接口，满足用户的应用需求。
- 算法调用层：包括算法调用的注册函数接口，用于回调算法，管理算法功能配置，内存分配，输入原始数据以及获取运算结果等。
- 算法库：包括 HR、SPO2 等算法库，它们将被算法调用层调用。

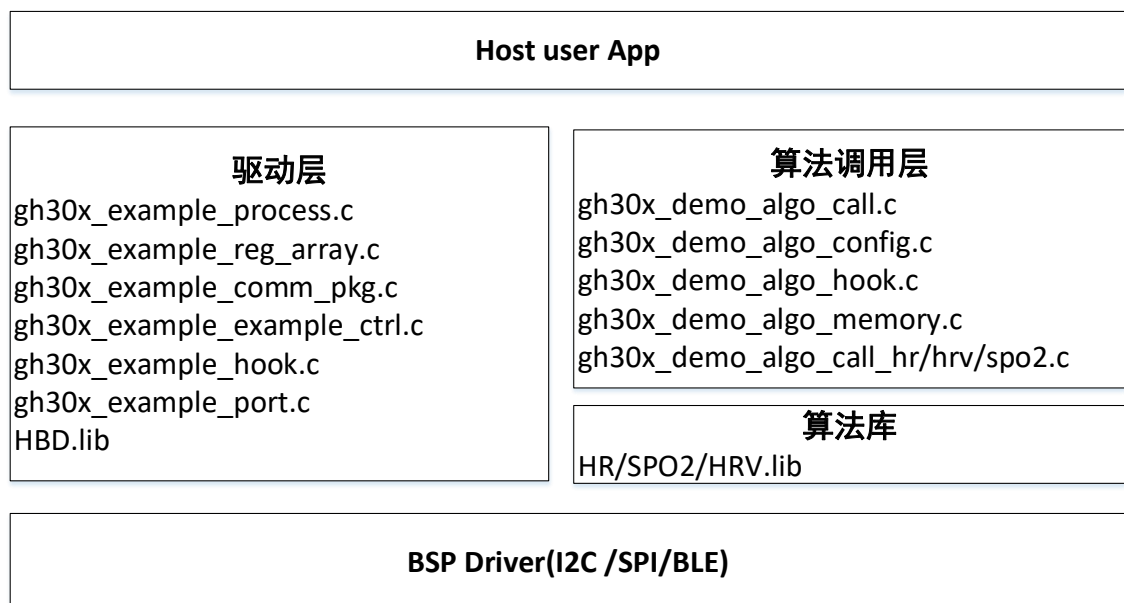


图 1-1 GH30x SDK 应用软件架构

说明：

- BSP 驱动层（BSP Driver）：用户平台的硬件接口驱动，可将功能接口注册到驱动层，供 SDK 操作 GH30x 芯片。

。

1.2 SDK 应用流程

GH30x SDK 应用流程主要包括三个子流程：上电初始化、G-sensor 中断处理、GH30x 模组中断处理。如下图所示：

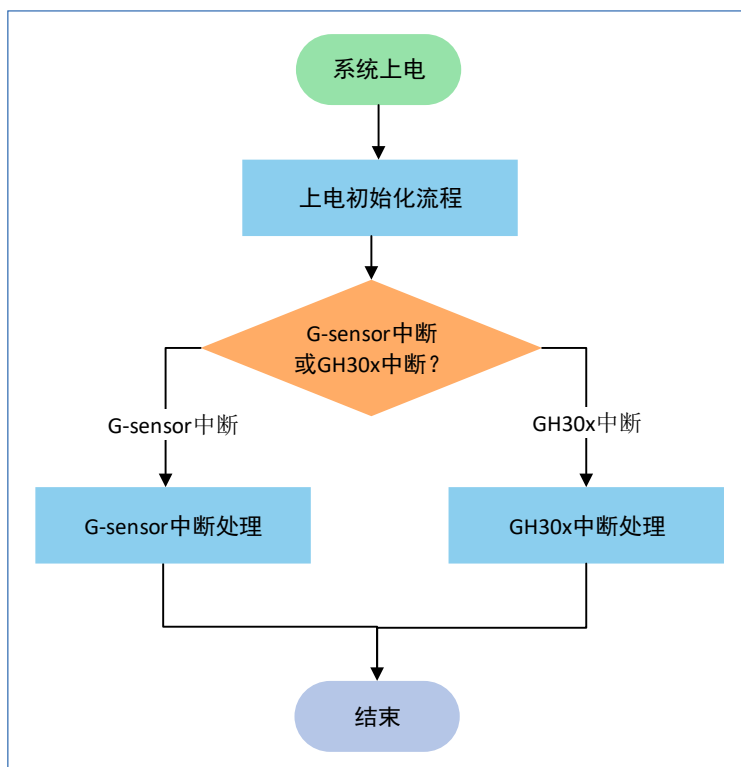


图 1-2 SDK 应用流程图

1. 上电初始化

系统上电后，先调用软件 init 接口先初始化 GH30x 模组，再初始化 G-sensor，具体流程如下图所示：

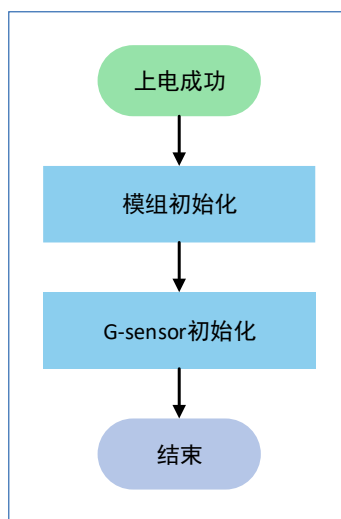


图 1-3 上电及初始化流程

2. G-sensor 中断处理

初始化完成后，可通过 G-sensor 中断处理流程读取 G-sensor 数据，进行动作检测，具体流程如下图所示：

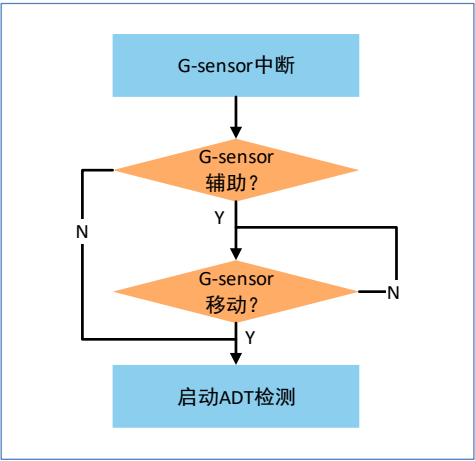


图 1-4 G-sensor 中断处理流程

1.3 驱动和算法调用层文件

GH30x SDK 包含各种 example 头文件及源文件、库文件等，具体描述参见下表：

表 1-1 GH30x SDK 文件说明

文件类型	文件名称	描述	备注
头文件 (.h)	gh30x_example.h	对外接口声明头文件, 提供给用户上层应用调用	
	gh30x_example_config.h	example 功能支持配置头文件	
	gh30x_example_common.h	example 公共头文件, 包含 example 通用的宏和常量定义, 以及函数声明等	用户只需关注产品形态 HBD_CTRL_LIB_DEV_VERSION 的定义
	gh30x_demo_algo_call.h gh30x_demo_algo_hook.h gh30x_demo_algo_memory.h gh30x_demo_algo_version.h gh30x_demo_algorithm_calc.h gh30x_demo_common.h	与算法调用.c 文件配合的头文件	
	gh30x_demo_algo_config.h	算法配置头文件	用户需要关注 GH30X_ALGO_FUNCTION_GROUP_0~4 这几个代表场景的宏, 根据实际的应用场景去配置, 这将会影响算法内存的分配, 详见 2.2.3 章节
	goodix_mem.h	算法内存管理头文件	
	goodix_type.h	算法数据类型头文件	
	goodix_hba.h	心率算法头文件	
	goodix_spo2.h	血氧算法头文件	

	goodix_hrv.h	心率变异性算法头文件	
驱动层 源文件 (.c) 和库文件	gh30x_example_port.c	example 功能移植接口源文件，包含各接口函数的实现代码	用户需根据平台进行移植适配
	gh30x_example_reg_array.c	初始化配置以及不同模式的寄存器配置	
	gh30x_example_comm_pkg.c	BLE/UART 数据打包函数接口定义	用户无需关注
	gh30x_example_ctrl.c	控制函数接口定义	
	gh30x_example_process.c	应用流程控制函数定义	
	gh30x_example_hook.c	驱动层回调函数定义	
	HBD.lib	驱动库	
算法调用 层 源文件(.c) 和库文件	gh30x_demo_algo_call.c	算法调用函数定义	
	gh30x_demo_algo_config.c	算法配置共用处理函数定义	
	gh30x_demo_algo_hook.c	算法结果回调函数定义	
	gh30x_demo_algo_memory.c	算法内存处理函数定义	
	gh30x_demo_algo_call_hr/hrv/spo2.c	各个算法调用函数定义	
算法库	common_cortexM4_keil-armcc-5.06.lib	算法公共库	
	hba_cortexM4_keil-armcc-5.06.lib	心率算法库	
	spo2_cortexM4_keil-armcc-5.06.lib	血氧算法库	
	HRV_KEIL5_M4-fp.lib	心率变异性算法库	
	NADT_KEIL5_M4-fp.lib	活体检测算法库	

一般情况下，移植 GH30x SDK 时，用户只需关注上述标红的文件

1.3.1 gh30x_example_config.h

gh30x_example_config.h 文件为 example 功能配置头文件，包含各种应用功能、芯片配置、初始化、算法功能等配置参数的宏定义，具体描述参见下表：

表 1-2 gh30x_example_config.h 文件配置说明

宏定义	描述
__GH30X_COMMUNICATION_INTERFACE__	芯片 IIC/SPI 接口选择(二选一)，需实现 gh30x_example_port.c 文件中对应接口函数
__PLATFORM_DELAY_US_CONFIG__	注册平台延时 μ s 函数使能，需在 gh30x_example_port.c 实现平台 μ s 延时函数 hal_gh30x_delay_us()
__GS_SENSITIVITY_CONFIG__	G-sensor 归一化精度配置，需与用户平台中的 G-sensor 精度相对应
__RETRY_MAX_CNT_CONFIG__	最大启动重试次数
__RESET_REINIT_CNT_CONFIG__	初始化重试次数
__FUNC_TYPE_ADT_ENABLE__	硬件 ADT 功能使能开关
__FUNC_TYPE_HR_ENABLE__	心率功能使能开关
__FUNC_TYPE_HRV_ENABLE__	HRV 功能使能开关
__FUNC_TYPE_SPO2_ENABLE__	血氧功能使能开关

__FUNC_TYPE_SOFT_ADT_ENABLE__	活体功能使能开关
__FUNC_TYPE_AGC_ENABLE__	AGC 功能使能开关
__USE_GOODIX_APP__	GHealth App 对接支持使能。若要上传 MCU 模式的 Debug 数据，则需开启 __ALGO_CALC_WITH_DBG_DATA__
__NEW_DATA_MULTI_PKG_NUM__	BLE 单次发送多包数据数量配置 配置多包发送支持时，需要用户平台的 BLE 支持长包发送
__BLE_PKG_SIZE_MAX__	BLE 发送的最长包的长度配置 该参数一般对齐 BLE MTU，并受限于手机支持的数据包长度
__BLE_MCU_PKG_BUFFER_MAX_LEN__	BLE 发送的 MCU 模式数据的 Buffer 长度 Buffer 长度需大于以下公式的计算值： (DBG_MCU_PKG_RAW_FRAME_LEN * __ALGO_CALC_DBG_BUFFER_LEN__) MCU_PKG_SPO2_ALGO_RESULT_LEN +
__UART_WITH_GOODIX_TOOLS__	UART 对应的 Goodix 量产测试工具使能
__SYSTEM_TEST_SUPPORT__	Local Mode 测试模式使能
__SYSTEM_TEST_DATA_CNT_CONFIG__	Local Mode 测试数据点数配置
__EXAMPLE_DEBUG_LOG_LVL__	调试信息等级配置
__EXAMPLE_LOG_DEBUG_SUP_LEN__	调试信息的最大字符串长度
__FACTORY_DET_SUPPORT__	测试模式使能
__HBD_CTR_WEAROFF_THRESHOLD__	产品形态为指夹仪时，需要根据实际情况设置调光脱落的阈值
__USER_DYNAMIC_ALGO_MEM_EN__	算法内存的申请形式，默认为 0 代表使用静态内存

1.3.2 gh30x_example.h

gh30x_example.h 文件为 example 应用的头文件，包含应用功能启动与停止、初始化等对外的函数接口，具体描述参见下表：

表 1-3 gh30x_example.h 文件说明

宏定义/函数声明	描述
gh30x_module_init	GH30x 模组初始化函数，上电/重新上电时必须调用
gh30x_module_start	GH30x 模组启动函数，根据模式启动 参 数 为 GH30X_FUNCTION_ADT/ GH30X_FUNCTION_HR/ GH30X_FUNCTION_HRV/ GH30X_FUNCTION_SPO2/ GH30X_FUNCTION_SOFT_ADT 的组合
gh30x_module_stop	GH30x 模块停止函数

1.3.3 gh30x_example_port.c

gh30x_example_port.c 文件包括 GH30x 芯片的硬件驱动接口，用于访问、操作 GH30x 的硬件模块，具体描述参见下表：

表 1-4 gh30x_example_port.c 文件说明

函数类型	函数名称	描述	备注
GH30x 通信接口函数	hal_gh30x_i2c_init	初始化 IIC 模块	IIC 与 SPI 通信方式二选一，且这些函数要求用户
	hal_gh30x_i2c_write	IIC 写操作	
	hal_gh30x_i2c_read	IIC 读操作	

	void hal_gh30x_spi_init	初始化 SPI 模块	必须实现 说明： IIC、SPI 数据格式要求，请参考对应的 GH30x 数据手册
	uint8_t hal_gh30x_spi_write	SPI 写操作	
	uint8_t hal_gh30x_spi_read	SPI 读操作	
	void hal_gh30x_spi_cs_set_low	设置 SPI 片选信号为低电平	
	void hal_gh30x_spi_cs_set_high	设置 SPI 片选信号为高电平	
G-sensor 驱动函数	gsensor_drv_init	G-sensor 初始化函数	用户可根据应用需求实现
	gsensor_drv_enter_normal_mode	G-sensor 配置为普通模式，可直接读取 x、y、z 寄存器获取数据	
	gsensor_drv_enter_fifo_mode	G-sensor 配置为 FIFO 模式，可通过 G-sensor FIFO 获取一组 x、y、z 数据	
	gsensor_drv_enter_motion_det_mode	G-sensor 配置为动作检测模式，检测动作输出中断	
	gsensor_drv_get_fifo_data	从 G-sensor FIFO 获取最新一组 x、y、z 数据，并缓存到 G-sensor 的软件 Buffer 中，缓存到软件 Buffer 中的数据对应为 GH30x 产生中断后的前 1s 数据	
	gsensor_drv_get_data	从 G-sensor 中获取 x、y、z 数据，获取的数据为最新一帧数据	
	gsensor_drv_int1_handler	G-sensor 中断处理函数	
延时函数	hal_gh30x_delay_us	μs 级延时函数	用户必须实现，且用户平台 SPI 需根据 Goodix 要求实现
用户平台 IO 中断初始化函数	hal_gh30x_int_init	用户平台与 GH30x INT 管脚连接的 IO 的外部中断初始化函数	GH30x 中断必须实现，G-sensor 中断可根据应用需求实现
	hal_gsensor_int1_init	用户平台与 G-sensor GINT_FIFO 管脚连接的 IO 的外部中断初始化函数	
	handle_wear_status_result	用户应用处理佩戴事件结果函数	
	handle_system_test_otp_check_result	用户应用处理量产测试 OTP 测试结果函数	
	handle_system_test_os_result	用户应用处理量产测试 OS 测试结果函数	
BLE 通信函数	gdcs_repeat_timeout_handler	分包定时发送函数，用户需要在此函数调用 gdcs_repeat_msg_handler，调用方式可以是直接调用也可以是发消息调用	对接用户应用心率 App 时实现
	ble_module_rcv_data_via_gdcs	BLE 接收数据处理函数，在用户平台接收 GHealth App 数据的函数中调用	用于对接 GHealth App，用户必须实现 说明： 关于自定义

	ble_module_send_data_via_gdcs	通过自定义的 GHealth App 服务发送数据	服务，可参考 2.3.2.1 新增蓝牙服务
定时器函数	ble_module_repeat_send_timer_start	BLE 重复发送数据定时器启动函数	BLE 重复发送数据定时器，需发送 MCU 数据时实现。 定时间隔取决于 BLE 稳定性，推荐 50 ~ 100 ms
	ble_module_repeat_send_timer_stop	BLE 重复发送数据定时器停止函数	
	ble_module_repeat_send_timer_init	BLE 重复发送数据定时器初始化函数	
	hal_gh30x_fifo_int_timeout_timer_start	FIFO 中断函数监控定时器启动函数	
	hal_gh30x_fifo_int_timeout_timer_stop	FIFO 中断函数监控定时器停止函数	
	hal_gh30x_fifo_int_timeout_timer_init	FIFO 中断函数监控定时器初始化函数	
UART 通信函数	uart_module_send_data	用户平台 UART 发送数据	对接 Goodix 量产工具 / Dongle 工具时需实现
	uart_module_rcv_data	UART 接收处理函数，在用户平台的 Goodix 量产工具 UART 数据接收函数中调用	
通信命令处理函数	handle_goodix_communicate_cmd	处理 Goodix 工具通信命令	处理 Goodix 工具命令时需实现
日志函数	example_dbg_log	用户平台输出 Log 函数	用户必须实现
动态内存函数	hal_gh30x_memory_malloc	用户平台动态内存申请接口实现	根据客户需要实现，打开宏 __USER_DYNAMIC_ALGO_MEM_EN_时必须实现
	hal_gh30x_memory_free	用户平台动态内存释放接口实现	根据客户需要实现，打开宏 __USER_DYNAMIC_ALGO_MEM_EN_时必须实现

1.3.4 gh30x_example_reg_array.c

gh30x_example_reg_array.c 包含各应用功能的配置数组。

表 1-5 gh30x_example_reg_array.c 文件说明

数组名	描述
hb_adt_confirm_reg_config	ADT 功能的配置
hb_reg_config_array	心率功能的配置
spo2_reg_config_array	血氧功能的配置
hrv_reg_config_array	心率变异性功能的配置

1.3.5 gh30x_example_hook.c

gh30x_example_hook.c 包含关键的驱动回调函数

函数名	描述
gh3x2x_get_rawdata_hook_func	驱动获取到原始数据的回调，详情见 API 说明章节
gh3x2x_algorithm_get_io_data_hook_func	驱动获取的功能帧数据的回调，详情见 API 说明章节

1.3.6 gh30x_demo_algo_config.h

gh30x_demo_algo_config.h 包含算法相关宏配置

宏	可配置值	描述
__HBD_ALGORITHM_VERSION__	见描述	可配置的算法版本： __HBD_ALGORITHM_VERSION_PRO__（资源占用大，性能最好） __HBD_ALGORITHM_VERSION_TINY__（资源占用一般，性能一般） __HBD_ALGORITHM_VERSION_BASIC__（资源占用最少，性能一般）
GH30X_ALGO_FUNCTION_GROUP_0	功能组 0	并行的算法组合 0 详情见“关于算法并行的说明”的章节
GH30X_ALGO_FUNCTION_GROUP_1	功能组 1	并行的算法组合 1 详情见“关于算法并行的说明”的章节
GH30X_ALGO_FUNCTION_GROUP_2	功能组 2	并行的算法组合 2 详情见“关于算法并行的说明”的章节
GH30X_ALGO_FUNCTION_GROUP_3	功能组 3	并行的算法组合 4 详情见“关于算法并行的说明”的章节
GH30X_ALGO_FUNCTION_GROUP_4	功能组 4	并行的算法组合 5 详情见“关于算法并行的说明”的章节
__USER_DYNAMIC_ALGO_MEM_EN__	0	静态算法内存池
	1	动态算法内存池
__USE_GOODIX_HR_ALGORITHM__	1	使用 GOODIX HR 算法
	0	不使用 GOODIX HR 算法
__USE_GOODIX_HRV_ALGORITHM__	1	使用 GOODIX HRV 算法
	0	不使用 GOODIX HRV 算法
__USE_GOODIX_SPO2_ALGORITHM__	1	使用 GOODIX SPO2 算法
	0	不使用 GOODIX SPO2 算法
__USE_GOODIX_SOFT_ADT_ALGORITHM__	1	使用 GOODIX SOFT_ADT 算法
	0	不使用 GOODIX SOFT_ADT 算法
__ALGO_LOG_CONFIG__	1	开启算法 log
	0	关闭算法 log

算法不同版本的基本信息表格：

算法版本	__HBD_ALGORITHM_VERSION_PRO__	__HBD_ALGORITHM_VERSION_TINY__	__HBD_ALGORITHM_VERSION_BASIC__
HR	资源占用最大，支持置信度分数与等级	无置信度	无置信度，资源占用最小
HRV	不分三个版本		
SPO2	公版带心率值输出	公版不带心率值输出	小资源版不带心率值输出
NADT	公版		小资源优化版

1.3.7 gh30x_demo_algo_hook.c

gh30x_demo_algo_hook.c 包含关键的算法计算回调函数

函数名	描述
GH30X_AdtAlgorithmResultReport	Adt 数据计算结果报告
GH30X_HrAlgorithmResultReport	心率算法计算结果报告
GH30X_Spo2AlgorithmResultReport	血氧算法计算结果报告
GH30X_HrvAlgorithmResultReport	HRV 算法计算结果报告
GH30X_SoftAdtAlgorithmResultReport	活体检测计算结果报告

2 例程移植说明

本章以 GH3011 为例介绍例程移植，其它型号（如 GH300、GH301、GH3018）可根据实际情况参考。

2.1 基本功能移植

本节以下列主控平台为例，介绍 GH30x SDK 基本功能（如 BLE 透传功能）的移植，以帮助用户快速入门 SDK 移植的操作流程。

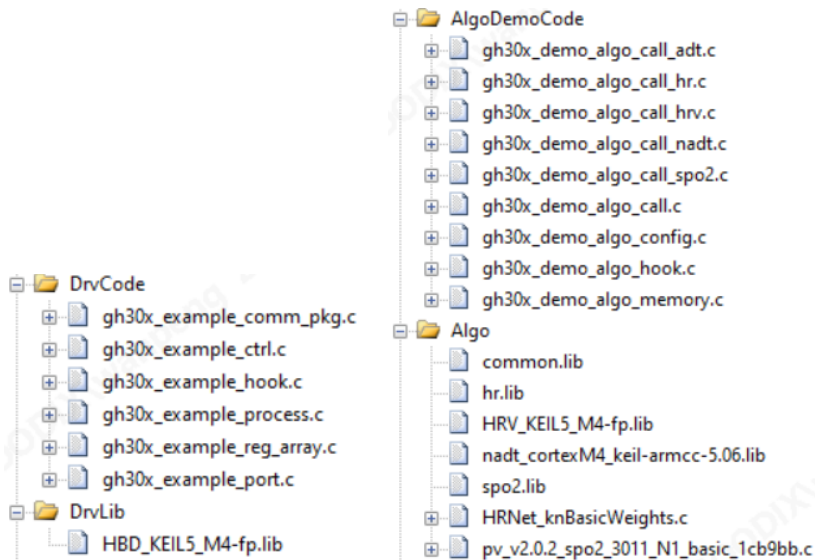
- 控制器内核：Cortex-M4
- 集成开发环境：Keil

 说明：

更多应用功能移植或调试，请参考 [2.2 应用方案介绍](#)和 [2.3 应用调试](#)。

2.1.1 移植 SDK 到工程

拷贝 GH30x SDK 文件至用户工程目录下，再打开 Keil 示例工程，添加的 SDK 文件（如图 2-1 所示）。



注：如果完全不使用 Goodix 的算法，可以将 HBD_KEIL5_M4-fp.lib 替换为 gh30x_hbd_ctrl.c。

图 2-1 驱动和算法移植到 keil 工程图

2.1.2 调通通信功能

用户可按照如下步骤，调通主控平台与 GH30x 模组间的通信功能：

1. 根据平台实现精准的 μs 级延时函数。

GH30x 的上电时序（如图 2-2 所示）明确地规定了上电后需要完成的初始化工作及精准的 μs 级延时。在 `gh30x_example_port.c` 文件中，用户需实现精准的 μs 级延时函数 `hal_gh30x_delay_us()`，具体实现方法可能因平台差异稍有不同。

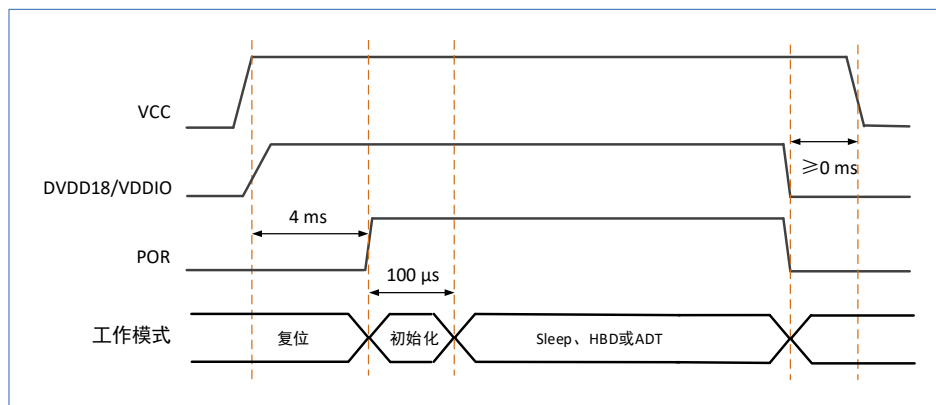


图 2-2 GH30x 上电时序图

2. 设置 IIC 通信。

- (1) 设置 IIC 管脚：将 IIC_EN 管脚配置为 IIC，并设置内部上拉（若外部硬件增加了上拉，可无需使用内部上拉）、IIC 通信速率以及 IIC 数据传输方式等。
- (2) 实现 IIC 读写函数：在 `gh30x_example_port.c` 文件中，实现 IIC 写函数 `hal_gh30x_i2c_write()` 和 IIC 读函数 `hal_gh30x_i2c_read()`。IIC 读函数需先利用 IIC 写函数写入一个数据，再进行读操作。对 GH30x 的所有操作均通过 IIC 读写函数实现，因此，需保证这两个函数的准确性。
- (3) 验证 IIC 通信：在应用工程中调用 `gh30x_module_init()` 函数，并利用 UART 打印信息验证 IIC 通

信是否正常。

- 如果 IIC 读写函数不正常或者 IIC 管脚设置不正确，UART 会打印出类似“gh30x init error[*]”的 Log 信息。
- 如果 IIC 读写正常并且 IIC 管脚设置正确，UART 会打印出类似“gh30x module init ok”的 Log 信息。
- 先发送 0XC0 让 Sensor 进入 IDLE 模式，延时 1ms 再直接读取寄存器 0x0028 的值，判断是否为 0x31。若寄存器值为 0x31，则表示 IIC 通信正常。

2.1.3 修改配置文件

如果用户应用是参照 Goodix 参考设计推荐值（参考对应的 GH30x 应用设计指南）进行设计的，可直接使用默认的配置文​​件。否则，需根据实际的应用设计及配置，修改配置文件 *gh30x_example_reg_array.c*，具体操作步骤如下：

1. 使用 EVK PC Tool 生成配置文件，如下图所示：

名称	修改日期	类型
☒ HRV_20220322200857.cleartext.conf 加密配置	2022/3/22 20:08	CONF 文件
☒ HRV_20220322200857.conf 未加密配置	2022/3/22 20:08	CONF 文件
☐ _HRV_20220322200857.sv	2022/3/22 20:08	SV 文件
☐ _穿戴确认ADTConfirm_20220322200857.cleartext.conf	2022/3/22 20:08	CONF 文件
☐ _穿戴确认ADTConfirm_20220322200857.conf	2022/3/22 20:08	CONF 文件
☐ _穿戴确认ADTConfirm_20220322200857.sv	2022/3/22 20:08	SV 文件
☐ _心率HB_20220322200857.cleartext.conf	2022/3/22 20:08	CONF 文件
☐ _心率HB_20220322200857.conf	2022/3/22 20:08	CONF 文件
☐ _心率HB_20220322200857.sv	2022/3/22 20:08	SV 文件
☐ _血氧SPO2_20220322200857.cleartext.conf	2022/3/22 20:08	CONF 文件
☐ _血氧SPO2_20220322200857.conf	2022/3/22 20:08	CONF 文件
☐ _血氧SPO2_20220322200857.sv	2022/3/22 20:08	SV 文件

图 2-3 EVK 工具生成的配置文件图

2. 按照下表所示的对应关系，将 EVK PC Tool 生成的配置文件中的数组内容替换到 *gh30x_example_reg_array.c* 文件中对应的驱动配置数组中。

表 2-1 EVK PC Tool 生成的配置文件与驱动配置的对应关系

配置文件	驱动配置（ <i>gh30x_example_reg_array.c</i> ）
_HRV_20220322200857.conf	hrv_reg_config_array
_穿戴确认 ADTConfirm_20220322200857.conf	hb_adt_confirm_reg_config
_血氧 SPO2_20220322200857.conf	spo2_reg_config_array
_心率 HB_20220322200857.conf	hb_reg_config_array

2.1.4 GH30X 的初始化函数的调用

初始化函数调用：gh30x_module_init，函数接口定义描述详见附录 API 说明

2.1.5 GH30X 的功能开启和关闭

功能开启：gh30x_module_start，函数描述详见 SDK API 说明章节

功能停止：gh30x_module_stop，函数描述详见 SDK API 说明章节

2.1.6 GH30X 中断处理函数的调用

中断处理分成上半部和下半部。上半部是在 INT 中断服务函数中完成，下半部为在任务线程或消息队列处理函数中完成。

上半部调用：hal_gh30x_int_handler_top_half

下半部调用：hal_gh30x_int_handler_bottom_half

2.1.7 验证测试

上述操作完成后，用户可按以下步骤验证 SDK 移植是否成功：

1. 编译示例工程，并将生成的应用固件文件下载至 BLE 设备。
2. 重启设备，运行程序。
3. 在手机端，使用蓝牙调试 App（GRTtoolbox）扫描并连接设备，查看是否发现添加的 GH30x 服务与特征。具体操作，可参考 2.3.2.5 验证蓝牙透传功能。
4. 蓝牙透传功能验证成功后，使用应用 App（GHealth App）连接设备，开启心率测试功能。如果在 App 界面查看到心率波形图、心率值，即表示 GH30x SDK 移植成功。

说明：

- Andriod 手机：支持蓝牙 4.0 及以上版本。
- 主控设备：支持蓝牙 4.0 及以上版本。
- 蓝牙调试 App：可使用 Goodix 提供的 GRTtoolbox App（Andriod），下载地址：
<https://product.goodix.com/zh/product/category/ble>。
- 应用 App：Goodix 提供的 GHealth App，可联系 Goodix 技术支持或代理商获取。

2.2 应用方案介绍

2.2.1 应用框图

GH3011 需要与蓝牙主板上的 MCU 进行数据交互，并搭配 G-sensor 使用（G-sensor 用于辅助心率检测算法，心率算法运行在主控 MCU 上）。绿光 LED 检测心率，红光 LED 加红外 LED 实现血氧检测，可选配红外 LED 进行佩戴检测。

GH3011 与 MCU 的通信方式可以选择 SPI 或 IIC 通信。另外，GH3011 提供一路 INT 中断信号，用于提示心率数据 Ready 和其他状态。在供电上，需要外部提供 VLED 及 VCC 供电电源。

GH3011 有 3 个 LED 驱动电流通道，同一个驱动通道上可并联多个 LED，型号必须相同，驱动通道的选择需要满足表 2-2 所示要求。

表 2-2 GH3011 LED 电流驱动通道选择

驱动通道	“血氧+心率”应用		“心率”应用	
	LED 光源	分配功能	LED 光源	分配功能
LED_DRV0	红外光	佩戴检测、血氧、夜间心率检测	红外光	佩戴检测、夜间心率检测

LED_DRV1	红光	血氧	绿光	心率
LED_DRV2	绿光	心率	绿光	

GH3011 典型应用的系统框图，如图 2-4（“血氧+心率”应用）和[错误!未找到引用源。](#)（“心率”应用）所示。

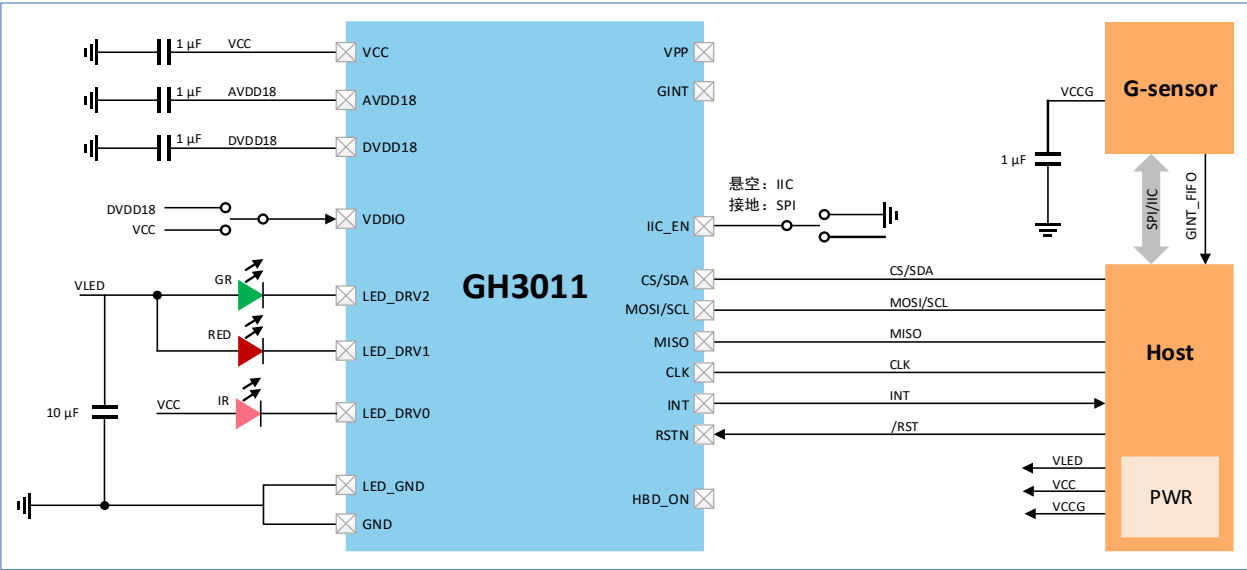
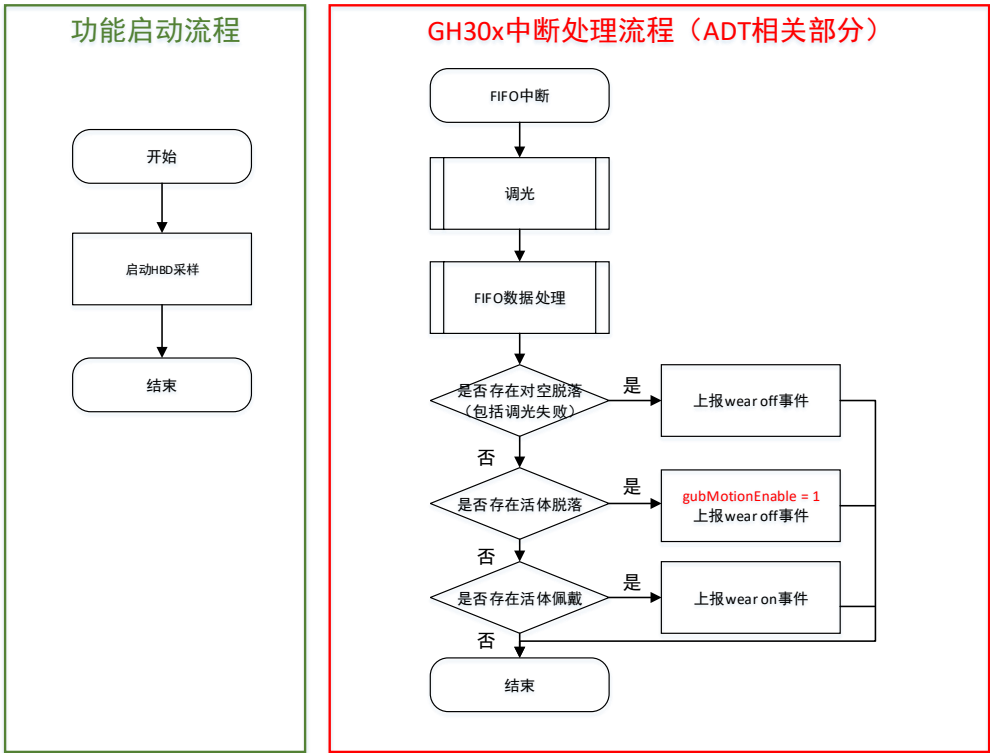


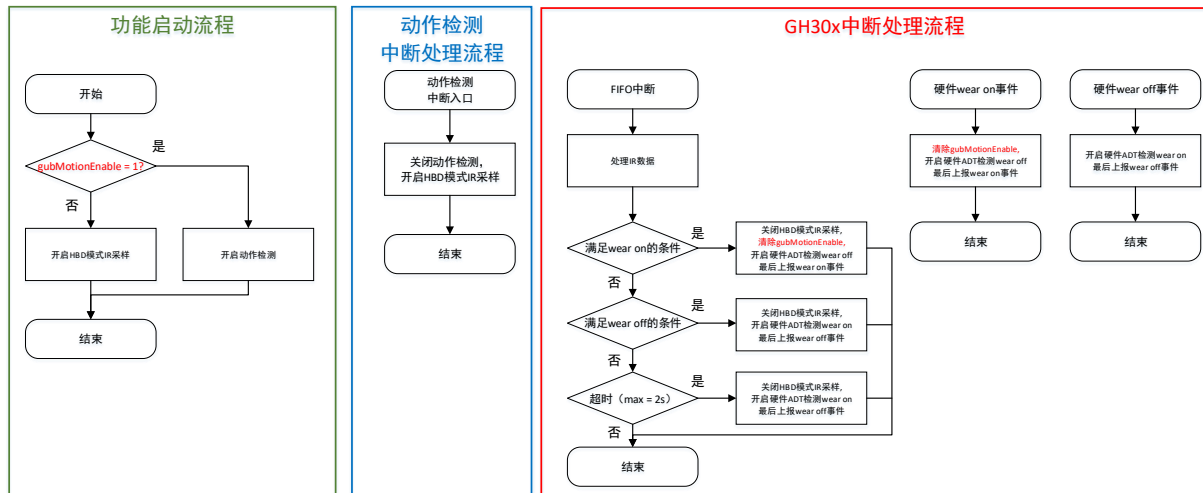
图 2-4 GH3011 典型心率及血氧检测应用方案系统框图

2.2.2 佩戴模块

开启 ADT+其他主功能（HR、SPO2 等）的流程



仅开启 ADT 的流程



2.2.3 关于算法并行的说明

算法调用层中允许设置5组功能，在 `gh30x_demo_algo_config.h` 中设置 GH30X_ALGO_FUNCTION_GROUP_0~4 的功能。

如在实际项目中：心率和活体检测同时运行，血氧单独运行，HRV 单独运行，则需要做如下设置：

```

#define GH30X_ALGO_FUNCTION_GROUP_0 (GH30X_FUNCTION_HR|GH30X_FUNCTION_SOFT_ADT)
#define GH30X_ALGO_FUNCTION_GROUP_1 (GH30X_FUNCTION_SPO2)
#define GH30X_ALGO_FUNCTION_GROUP_2 (GH30X_FUNCTION_HRV)
#define GH30X_ALGO_FUNCTION_GROUP_3 (GH30X_NO_FUNCTION)
#define GH30X_ALGO_FUNCTION_GROUP_4 (GH30X_NO_FUNCTION)
  
```

注意：

1、不同的设置，会影响到算法内存池大小，其计算公式为：

各组分别计算：

总常驻内存 = 功能1常驻内存 + 功能2常驻内存... + 功能n常驻内存

峰值内存差值 = MAX[各功能峰值内存 - 各功能常驻内存]

余量 = MAX[各功能余量]

GH3X2X_ALGORITHMS_MEMORY_SIZE_FINAL_0~4 = 总常驻内存+峰值内存差值+余量

GH3X2X_ALGORITHMS_MEMORY_SIZE_FINAL = MAX[GH3X2X_ALGORITHMS_MEMORY_SIZE_FINAL_0~4]

2、血氧算法与活体检测算法不能同时开启，因为 Gh30x 只支持同时采集两路数据，血氧算法需要红灯与红外灯，活体检测算法需要绿灯，所以不能同时开启。

2.3 应用调试

2.3.1 G-sensor 调试

2.3.1.1 G-sensor 要求

为保证动态心率算法的正常工作，G-sensor 必须满足以下4项参数要求：

表 2-3 G-sensor 要求

参数	要求	备注
----	----	----

精度	512LSB/g	若 G-sensor 精度不是 512LSB/g，则需进行精度归一化处理。传递给心率算法的精度必须为 512LSB/g。
量程与跳点	量程 $\geq \pm 4g$ ，无跳点	建议量程设置为 $\pm 4g$
噪声	不大于 25LSB	
采样周期	$\geq GH30x$ 的采样周期	心率算法要求送入的数据量为前 1s 的 G-sensor 数据，并要求 G-sensor 的采样频率为 25 Hz。 例如，若 G-sensor 的采样频率为 25 Hz，则送入的数据量为 25 个；若 G-sensor 的采样频率为 50 Hz，则送入的数据量为 50。 如果 25 Hz 采样频率送入的数据量不足 25 个，需对 G-sensor 数据进行补值。 提示：如果开发前期没有准备好 Gsensor，可以传入随机值填充（随机值符合高斯分布时效果较佳），下图会展示一段 C 语言实现的示例代码。
<pre> #include "stdlib.h" #include "goodix_type.h" #define MAX(X, Y) ((X) >= (Y) ? (X) : (Y)) #define MIN(X, Y) ((X) >= (Y) ? (Y) : (X)) void Gh30xRandomGsensorDataGenerate(GS16 *pGsensorData, GU32 unDataSize, GU32 unSeed) { GS16 *pData = pGsensorData; GU32 unCnt = unDataSize; srand(unSeed); while (unCnt--) { *pData = (GS16)(MAX(-10, MIN(10, (((float32_t)rand() / RAND_MAX) * 3)))); pData++; } } </pre>		

2.3.1.2 G-sensor 驱动调试

为调试 G-sensor 驱动，用户需在 *gh30x_example_prot.c* 文件中实现 G-sensor 初始化、通用模式、FIFO 模式、动作检测模式等接口函数。

- G-sensor 初始化：gsensor_drv_init（必须实现）

gsensor_drv_init()函数用于初始化 G-sensor 参数，要求用户必须实现。

```

int8_t gsensor_drv_init(void)
{
    int8_t ret = GH30X_EXAMPLE_OK_VAL;
    gsensor_drv_motion_det_mode = false;
    // code implement by user
    /* if enable all func equal 25Hz, should config > 25Hz;
    but if enable have 100hz, should config to > 100hz. if not, feedback to GOODIX!!!

```

```
*/  
return ret;  
}
```

说明:

若用户已在其他模块实现了该函数，可替换 *gh30x_example_prot.c* 中的示例代码。

- 通用模式: `gsensor_drv_enter_normal_mode`

`gsensor_drv_enter_normal_mode()`函数用于使能 G-sensor 低功耗模式。在该模式下，G-sensor 不会自动采集数据。

```
void gsensor_drv_enter_normal_mode(void)  
{  
    // code implement by user  
    gsensor_drv_motion_det_mode = false;  
}
```

- FIFO 模式: `gsensor_drv_enter_fifo_mode`

`gsensor_drv_enter_fifo_mode()`函数用于使能 G-sensor 测量模式。在该模式下，G-sensor 将测量的数据存放在 FIFO 中。通过切换 `gsensor_drv_enter_normal_mode` 和 `gsensor_drv_enter_fifo_mode`，可降低功耗。若要和其他功能共用 G-sensor，仅需保证能够按时、按量传入 G-sensor 数据到算法接口即可。

```
void gsensor_drv_enter_fifo_mode(void)  
{  
    // code implement by user  
    gsensor_drv_motion_det_mode = false;  
}
```

- 动作检测接口: `gsensor_drv_enter_motion_det_mode`

`gsensor_enter_motion_det_mode()`函数用于使能 G-sensor 动作检测模式。在该模式下，G-sensor 将进行运动检测。若检测到运动超过设定阈值，则将调用中断服务，触发 GH30x 检测。

```
void gsensor_drv_enter_motion_det_mode(void)  
{  
    // code implement by user  
    gsensor_drv_motion_det_mode = true;  
}
```

说明:

若应用逻辑无需检测功能，则可在 *gh30x_example_config.h* 文件中，将 `__HB_START_WITH_GSENSOR_MOTION__` 设置为“0”，且无需实现该函数。

- G-sensor 获取寄存器数据、FIFO 数据的接口：
 - 获取寄存器数据: `gsensor_drv_get_data`（一般用于透传）
 - 获取 FIFO 数据: `gsensor_drv_get_fifo_data`（用于 MCU 端传送 G-sensor 数据给算法，必须实现）。

2.3.1.3 G-sensor 应用场景

G-sensor 主要应用于佩戴检测和动态心率检测。

- 佩戴状态检测

佩戴状态检测流程包括：佩戴检测和脱落检测，如图 2-5 所示。

在未佩戴状态下，由 G-sensor 动作触发佩戴检测；在佩戴状态下，通过活体检测判断是否脱落。

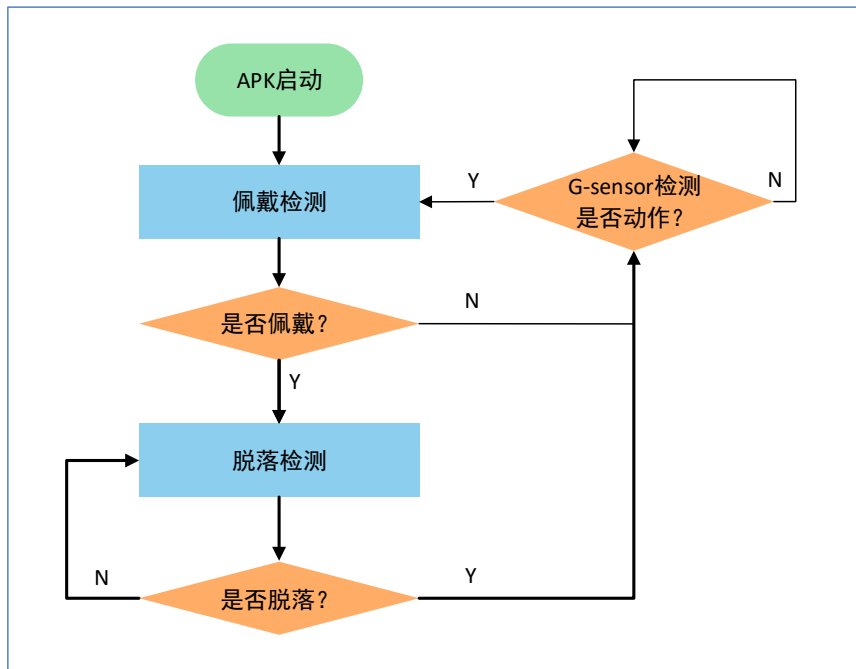


图 2-5 佩戴状态检测流程

1. 佩戴检测

佩戴检测包含接近检测和佩戴确认：

- (1) 接近检测以 5 Hz 的频率采集红外 PPG 数据。
- (2) 佩戴确认采样频率为 25 Hz。通过比较 G-sensor 幅度/PPG 与阈值，判断是否为佩戴状态。

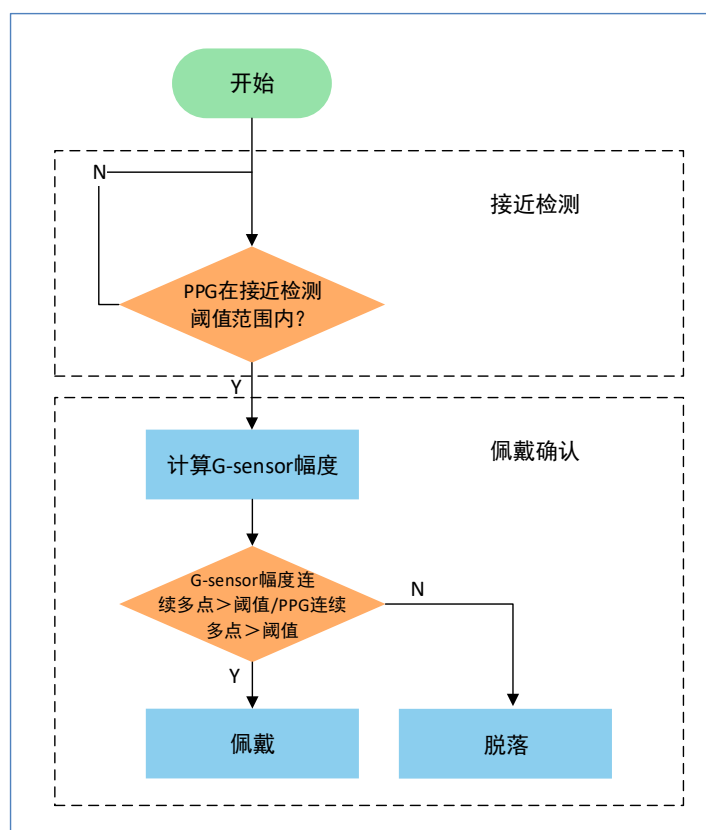


图 2-6 接近检测和佩戴确认流程

2. 脱落检测

通过判断 G-sensor 加速度是否变化，或 PPG 信号是否正常，进行脱落检测。

检测周期为 1s。

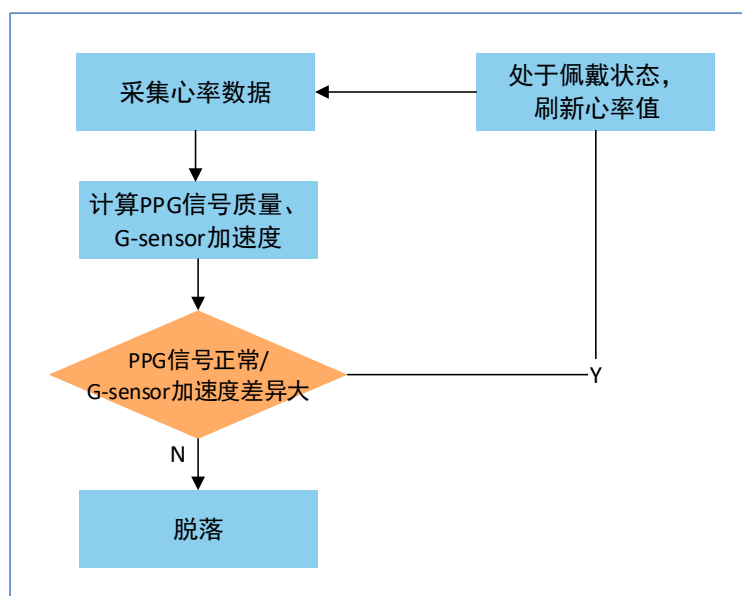


图 2-7 脱落检测流程

- 心率检测

心率算法将利用 G-sensor 数据进行运动的判断和去除运动噪声等。

2.3.1.4 G-sensor 应用设置

参考表 2-3，心率算法对 G-sensor 的精度要求为 512 LSB/g，量程 $\geq \pm 4g$ 。若不符合要求，则需进行调整。G-sensor 数据宽度的计算公式为：

$$2^x = \text{LSB} * \text{满量程} \quad (\text{比如} \pm 4g \text{ 的满量程是 } 8) \quad (1)$$

由上述公式，可计算出 G-sensor 的最小数据宽度为 $512 * 8 (\pm 4) = 4096$ ，即 2^{12} 。因此，G-sensor 数据位宽为 12-bit。

当 G-sensor 精度小于 512LSB/g 或量程小于 4g 时，需根据 G-sensor 实际情况配置参数或更换 G-sensor；当 G-sensor 位宽大于 12-bit 时，需进行归一化处理，以使得 G-sensor 精度满足算法要求。

在 *hbd_ctrl.h* 中，定义了 G-sensor 精度的预处理宏。

```
typedef enum
{
    HBD_GSENSOR_SENSITIVITY_512_COUNTS_PER_G = 0,
    HBD_GSENSOR_SENSITIVITY_1024_COUNTS_PER_G,
    HBD_GSENSOR_SENSITIVITY_2048_COUNTS_PER_G,
    HBD_GSENSOR_SENSITIVITY_4096_COUNTS_PER_G,
    HBD_GSENSOR_SENSITIVITY_8192_COUNTS_PER_G,
} EM_HBD_GSENSOR_SENSITIVITY;
```

G-sensor 数据经过精度预处理后，再传入心率算法。

```
GS8    HBD_HbCalculateByFifoIntEx(ST_GS_DATA_TYPE  stGsAxisValue[],    GU16    usGsDataNum,
EM_HBD_GSENSOR_SENSITIVITY    emGsensorSensitivity,    GS32    nRawdataOut[][6],    GU16
*pusRawdataOutLen, ST_HB_RES  *pstHbRes)
```

用户可在 *gh30x_example_config.h* 中修改 G-sensor 精度的宏定义。

例如，若 G-sensor 实际精度为 2048LSB/g，则可按下列代码设置宏定义：

```
/// gsensor sensitivity normalized
#define __GS_SENSITIVITY_CONFIG__    (HBD_GSENSOR_SENSITIVITY_2048_COUNTS_PER_G)
```

或者，在送入算法前对 G-sensor 数据右移 2 位处理。

G-sensor 数据对齐

心率算法获取 G-sensor 数据的目的是消除运动干扰等。因此，G-sensor 数据和 GH30x 的采样需保持同步。心率算法的处理逻辑为：主控接收到 GH30x 中断后，即给算法传入 GH30x 的 1s 数据。因此，为了获取同步的 G-sensor 数据，主控接收到 GH30x 中断后，需从 G-sensor Buffer 中读取前 1s 的数据。

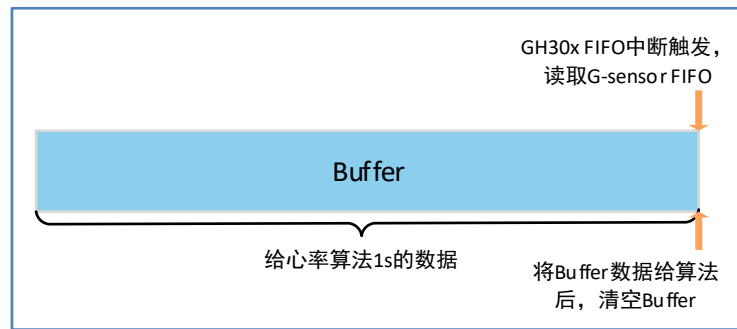


图 2-8 G-sensor 数据对齐

用户可通过定义一个软件 Buffer，实现 G-sensor 数据对齐。当 GINT_FIFO 管脚（主控设备与 G-sensor 连接的中断管脚）连接或断开时，定义的软件 Buffer 结构有所不同。

- 连接 GINT_FIFO

在连接 GINT_FIFO 的情况下，用户需定义一个软件 Buffer，其结构如下图所示。

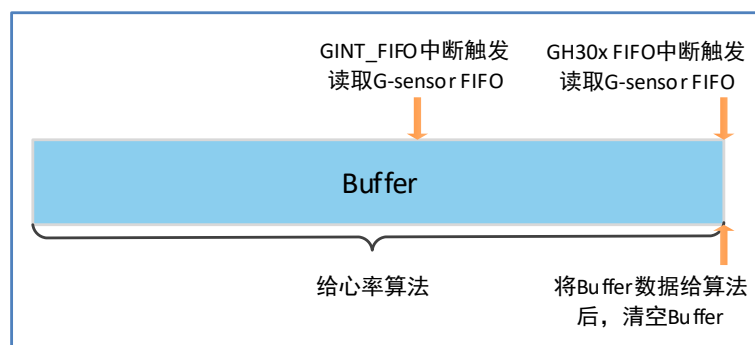


图 2-9 Buffer 结构图

中断处理包括两种：GINT_FIFO 中断和 GH30x FIFO 中断。

- GINT_FIFO 中断触发读取 G-sensor FIFO:

读取 G-sensor FIFO 数据，写入 Buffer，改变 Index。

- GH30x FIFO 中断触发读取 G-sensor FIFO:

读取 G-sensor FIFO 数据，写入 Buffer，改变 Index。

将主控从 Buffer 读取的数据发送给 HBD_HbCalculateByFifoInt 进行计算，Index 设置为 0。

说明：

上述为推荐方案，具体实现可参考示例工程以及示例代码。

- 断开 GINT_FIFO:

在断开 GINT_FIFO 管脚的情况下，用户需定义一个软件 Ring Buffer，其结构如下图所示。

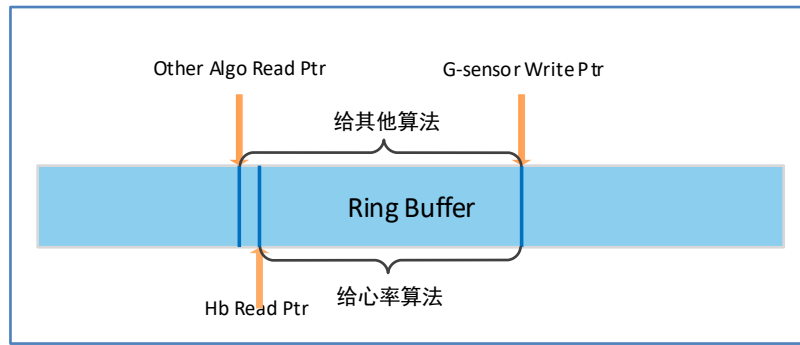


图 2-10 Ring Buffer 结构图

说明：

如果 G-sensor 数据还有其他应用也可使用该方式，一个写指针，不同应用对应不同读取指针，对应的应用获取的数据为对应读取指针与写指针间的数据。

中断处理包括两种：定时读取 G-sensor FIFO 的中断，GH30x FIFO 中断。

◦ 定时器中断触发读取 G-sensor FIFO：

- (1) 读取 G-sensor FIFO 数据，写入 Ring Buffer，改变 G-sensor write ptr。
- (2) 若存在其他应用，主控先将 Other App Read Ptr 和 G-sensor Write Ptr 之间的数据发送给其他应用，再将 Other App Read Ptr 赋值为 G-sensor write ptr。

◦ GH30x FIFO 中断触发读取 G-sensor：

- (1) 读取 G-sensor FIFO 的数据，写入 Ring Buffer，改变 G-sensor write ptr。
- (2) 读取 Ring Buffer 数据，将 Hb read ptr 赋值为 G-sensor write ptr，读取的数据会发送给 HBD_HbCalculateByFifoIntEx()函数进行计算。

时间轴示例参见图 2-11。其中，定时时间为 500 ms，G-sensor 采样率为 50 Hz。

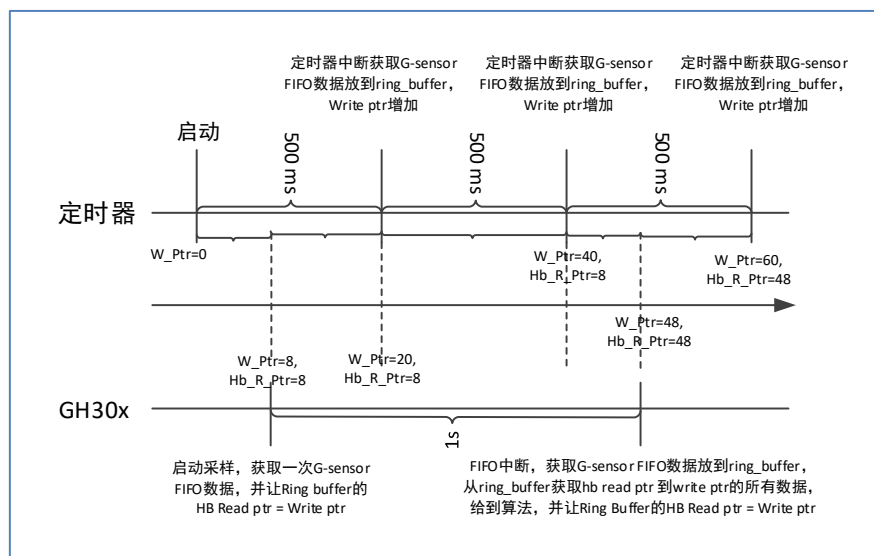


图 2-11 时间轴示例

2.3.1.5 G-sensor 应用配置

脱落后再次进行佩戴检测时，如果有开启 Software ADT 功能，则会在活体脱落后自动开启 Gsensor 运动检测。

Gsensor 运动检测需要用户实现 `gsensor_drv_enter_motion_det_mode()` 函数，并注册 G-sensor 触发回调函数 `hal_gsensor_drv_int1_handler()`。

/// 具体可见 `g_uchGh30xMotionDetEn` 变量的使用

2.3.2 蓝牙透传调试

对于具有蓝牙功能的应用，可利用 App 软件，将主控设备的数据通过蓝牙（BLE）传输到手机，进行蓝牙透传功能调试。

说明：

- Andriod 手机：支持蓝牙 4.0 及以上版本。
- 主控设备：支持蓝牙 4.0 及以上版本。
- 蓝牙调试 App：可使用 Goodix 提供的 GRTtoolbox App（Andriod），下载地址：
<https://product.goodix.com/zh/product/category/ble>。
- 应用 App：Goodix 提供的 GHealth App，可联系 Goodix 技术支持或代理商获取。

下面章节将介绍蓝牙透传调试的具体操作步骤。

2.3.2.1 新增蓝牙服务

在蓝牙主控芯片的 BLE GATT 初始化代码中，添加一个自定义服务和两个特征，具体要求如下：

- 自定义服务 UUID: "0000190e-0000-1000-8000-00805f9b34fb"
GHealth App 专用 BLE 服务，用于传输算法数据（设备与手机间的通信）
- 发送特征 UUID: "00000003-0000-1000-8000-00805f9b34fb"
GHealth App 发送特征，用于设备向手机上传算法相关数据。
- 接收特征 UUID: "00000004-0000-1000-8000-00805f9b34fb"
GHealth App 接收特征，用于手机向设备下发算法相关命令。

对于满足要求（参考[错误!未找到引用源。](#)[错误!未找到引用源。](#)）的蓝牙主控平台，均可添加上述自定义服务与特征。

为方便操作，用户可直接将下列代码拷贝到对应的 GATT 初始化函数中。

```
#define GH30X_SERVICE_UUID {0xFB, 0x34, 0x9B, 0x5f, 0x80, 0x00, 0x00, 0x80, 0x00, 0x10, 0x00, 0x00, 0x00, 0x0E, 0x19, 0x00, 0x00}
#define GH30X_TX_UUID {0xFB, 0x34, 0x9B, 0x5f, 0x80, 0x00, 0x00, 0x80, 0x00, 0x10, 0x00, 0x00, 0x03, 0x00, 0x00, 0x00}
#define GH30X_RX_UUID {0xFB, 0x34, 0x9B, 0x5f, 0x80, 0x00, 0x00, 0x80, 0x00, 0x10, 0x00, 0x00, 0x04, 0x00, 0x00, 0x00}
```

代码添加完成后，可利用蓝牙调试 App（GRTtoolbox）进行测试验证。若在服务列表中，发现自定义的服务和特征（如图 2-12 所示），即表示成功新增蓝牙服务。

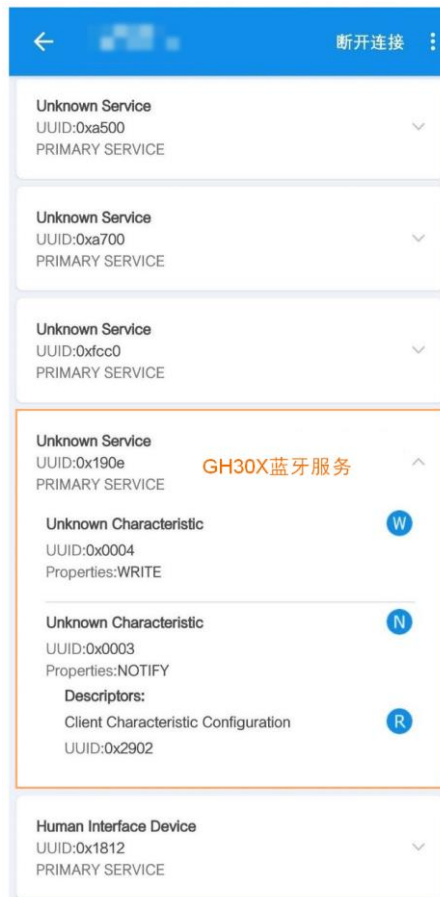


图 2-12 服务初始化界面

2.3.2.2 注册蓝牙接收数据回调函数

GHealth App 连接设备后，会寻找所需的蓝牙服务和特征；成功找到后，再通过 GHealth App 接收特征下发命令；此时，BLE 主控软件将触发通知事件。

用户代码需调用蓝牙数据解析函数 `ble_module_recv_data_via_gdcs()`，如下所示：

```
void ble_module_recv_data_via_gdcs(uint8_t *data, uint8_t length)
{
    gh30x_app_cmd_parse(data, length);
}
```

说明：

- 函数声明位于 `gh30x_example.h`，需确保将该文件加入工程并参与编译。
- 函数实现代码位于 `gh30x_example_port.c`，需确保该文件加入工程头文件的搜索目录。

2.3.2.3 注册蓝牙发送数据回调函数

设备接收 GHealth App 发送的数据后，需进行回复。此时，算法库会自动调用注册的蓝牙发送函数，再进行数据上传操作。在 GH30x 算法库初始化阶段，调用 `gh30x_comm_pkg_init()` 函数将 BLE 发送函数注册到算法库中。

当注册的蓝牙发送函数需发送数据时，可调用 `gh30x_example_port.c` 文件中的 `ble_module_send_data_via_gdcs()` 函数，其示例代码如下：

```
uint8_t ble_module_send_data_via_gdcs(uint8_t data[], uint8_t length)
{
    ...
}
```

```
uint8_t ret = GH30X_EXAMPLE_OK_VAL;
// code implement by user
ble_tx_data_send(data, length);
return ret;
}
```

用户需根据实际情况，添加蓝牙主控平台的发送数据函数的调用（替换示例代码中的绿色语句）。

2.3.2.4 注册 MCU 模式发送数据定时器函数

在蓝牙 MCU 模式下，需使用一个定时器作为事件驱动发送数据。在 GH30x 算法库初始化阶段，调用 gh30x_comm_pkg_init() 函数初始化定时器。

用户需在 gh30x_example_port.c 文件的 ble_module_repeat_send_timer_init() 函数中添加用户代码，以实现定时器初始化。定时器建议配置为 50 毫秒触发，且蓝牙连接间隔小于定时器触发时间 50 毫秒。

```
void ble_module_repeat_send_timer_init(void)
{
    // code implement by user
    // must register func ble_module_repeat_send_timer_handler as callback
    /* should setup 100ms timer and ble connect interval should < 100ms
    */
}
```

另外，用户还需实现开启定时器、关闭定时器这两个函数，并且在定时器触发函数中增加发送数据函数 ble_module_repeat_send_timer_handler() 的调用，参考代码如下：

```
void ble_module_repeat_send_timer_start(void)
{
    // code implement by user
}
void ble_module_repeat_send_timer_stop(void)
{
    // code implement by user
}
void TIMER_IRQHandler(void)
{
    ble_module_repeat_send_timer_handler();
}
```

2.3.2.5 验证蓝牙透传功能

上述操作完成后，可使用蓝牙调试 App（以 GRTtoolbox App 为例）验证蓝牙透传功能。

1. 将设备正常佩戴在合适位置，并使用 GRTtoolbox 连接设备。
2. 当 GRTtoolbox 成功发现服务后，进入服务列表页面。找到 GHealth App 服务，然后打开发送特征的通知功能，如图 2-13 的图 1 所示。
3. 使用接收特征写入四字节的十六进制数据“0xC022452C”，并发送给主控设备，如图 2-13 的图 2 所示。随后，主控设备的绿灯将被点亮并闪烁。

4. 使用发送特征接收数据，先接收到上一步发送的数据“0xC022452C”，随后持续接收数据，如图 2-13 的图 3 所示，即表示蓝牙透传功能已正常工作。

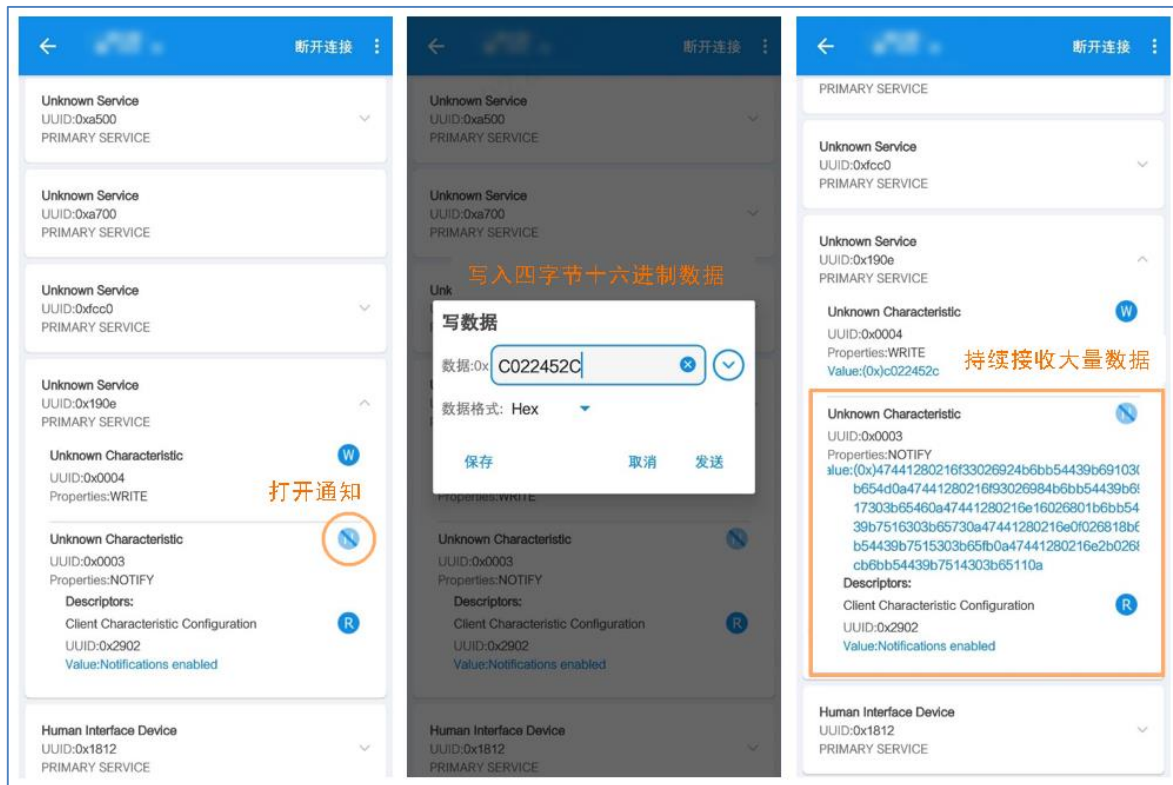


图 2-13 蓝牙透传功能验证 App 界面

在蓝牙透传 MCU 模式下，使用设备固件中的算法库执行运算。算法执行完毕后，将运算结果和参与运算的 PPG、ACC 源数据打包，再通过蓝牙发送至手机端。通常每秒开启一轮蓝牙发送，并需在下次蓝牙发送开启前，完成所有数据的发送：心率模式每秒发送 650 字节左右，血氧模式每秒发送 2600 字节左右。蓝牙透传 MCU 模式，需配合 [错误!未找到引用源。错误!未找到引用源。](#) 初始化的定时器一起使用。

说明：

App 仅作为数据接收者，不会对设备固件运行产生影响，可以最大限度地做到与终端用户使用效果一致。

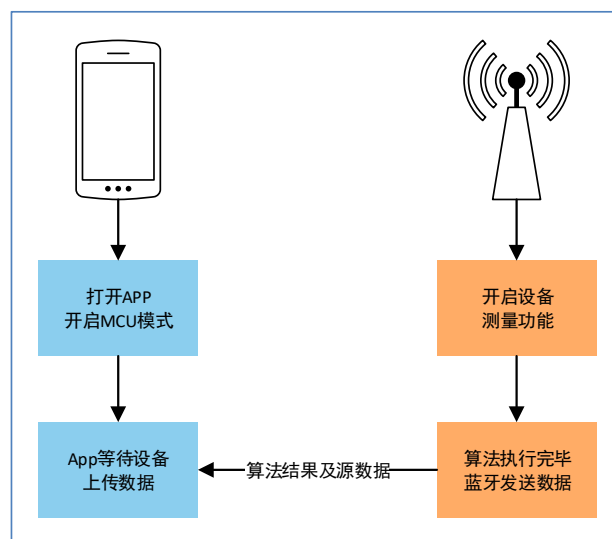


图 2-14 蓝牙透传调试应用模式

2.3.2.6 配置蓝牙透传功能

- `__USE_GOODIX_APP__`: 使用蓝牙透传调试功能连接 GHealth App 时, 需开启该宏定义。用户可在调试版本中开启该宏定义, 在发布版本中关闭它。
- `__NEW_DATA_MULTI_PKG_NUM__`: 使用蓝牙透传功能在一次调用数据发送函数时拼包发送多个数据包, 该宏定义数量加 1 为拼包数量, 默认值为 4, 表示每次拼 5 包数据, 共计 $5 \times 22 = 110$ 字节。
- `__BLE_PKG_SIZE_MAX__`: 使用蓝牙发送最长数据包定义, 一般对齐蓝牙连接后更新的 MTU, 该定义与手机蓝牙强相关, 需根据手机支持情况再合理设置。
- `__BLE_MCU_PKG_BUFFER_MAX_LEN__`: 蓝牙透传 MCU 模式下的数据缓冲区长度, 其长度值应大于该公式计算值: $(DBG_MCU_PKG_RAW_FRAME_LEN * __ALGO_CALC_DBG_BUFFER_LEN__) + MCU_PKG_SPO2_ALGO_RESULT_LEN$ 。
- `DBG_MCU_MODE_PKG_LEN`: 蓝牙 MCU 模式数据包个数, 保持默认不修改。
- `DBG_MCU_PKG_RAW_FRAME_LEN`: 蓝牙 MCU 模式数据帧长度, 保持默认不修改。
- `DBG_MCU_PKG_HEADER_LEN`: 蓝牙 MCU 模式数据包头长度, 保持默认不修改。

注意:

1. 使用蓝牙透传调试功能时, 请确认蓝牙连接间隔(Connection Interval)设置是否可以满足数据传输使用。App 模式心率应用要求连接间隔小于 40 毫秒, 血氧应用要求连接间隔小于 10 毫秒; MCU 模式心率应用要求每秒可以发送 650 字节数据, MCU 血氧应用要求每秒可以发送 2600 字节数据。
2. 使用蓝牙 MCU 模式, 需要确认蓝牙 MTU 长度, 避免应用层单次发送的数据长度大于蓝牙最大单次发送的数据长度, 造成堵塞影响蓝牙芯片正常工作。

2.3.3 量产测试调试

量产测试测试, 一般是在采用 MCU 本地测试和 MT 测试 (或者整机测试) 中二选一, MCU 本地产测是不是必须的, 视用户实际应用情况而定。

2.3.3.1 MCU 本地测试

测试准备

如果用户确定使用 MCU 本地测试, 测试之前需完成以下准备工作:

- 硬件设备: GH30x 模组、乳白色测试头、黑色测试头。
- 产测库: GH30x 产测库 (TestLib v1.1.0 及以上版本)
- 使用 MCU 本地产测功能时, 需关注的使能开关和测试接口包括:
 - `__SYSTEM_TEST_SUPPORT__`: MCU 本地产测使能。
 - `gh30x_systemtest_start`: 启动 MCU 本地产测的接口函数。

一般情况下, MCU 本地测试包括放置白色测试头的光路灵敏度测试和放置黑色吸光硅胶头的漏光测试。由于漏光和灵敏度测试的物理要求不一样, 所以可利用物理按键触发漏光和灵敏度测试事件。

测试步骤

测试准备工作完成后, 可按如下步骤开始测试:

1. 在 `gh3011_example_config.c` 中，将宏定义 `__SYSTEM_TEST_SUPPORT__` 配置为 1，使能 MCU 本地产测功能。
2. 在 `main` 函数中调用 `gh30x_systemtest_start(EMGh30xTestItem mode)` 函数，并选择测试模式：
 - `HBDTEST_TESTIEM_COMM` 通信测试
 - `HBDTEST_TESTIEM_OTP` OTP 测试
 - `HBDTEST_TESTIEM_CTR` 光路灵敏度测试（包括噪声测试）
 - `HBDTEST_TESTIEM_LEAK` 漏光测试
 - `HBDTEST_ONLYTEST_CTR_LEAK` 光路灵敏度测试和漏光测试
 - `HBDTEST_TEST_ALL_TEST` 通信测试、OTP 测试、光路灵敏度测试、漏光测试
3. 在 `gh3011_example_reg_array.c` 中，配置 `ledmask` 选择 LED 0-2 进行测试，`ledmask[0]-ledmask[2]` 分别对应 LED 0-1，相应 LED 置 1，即表示使用该 LED 进行测试。通过配置 `led0std/led1std/led2std`，可设置对应 LED 的测量阈值和其他默认值，若无需默认值，则置 0 即可。在测试过程中，如有测试不通过，则会立即停止测试，可以通过打印信息查看 CTR、LEAK、测试不通过的 LED 等结果。可参考如下的 `HBDTEST_ROMATSTRes` 结构体定义或头文件 `systemTest.h`。

结构体 `HBDTEST_ROMATSTRes`：

- `_flag`：若该变量置为 1，则产测功能会去计算 `_leadRatio`
- `_CTR`：CTR 测试结果
- `_leak`：漏光测试结果
- `_leakRatio`：Leak/CTR
- `_noise`：噪声测试结果

`HBDTEST_ROMATSTRes` 结构体为测试计算结果，也可在 `gh3011_example_reg_array.c` 设置相关默认值。若 `_flag` 为 0，则 `leakRatio` 无意义，将一直为 0，如：只进行漏光测试模式；若在 `gh3011_example_reg_array.c` 里设置默认值，当 `_flag` 为 1 时，`_CTR` 不能设置为 0。

```
HBDTEST_ROMAStd: _ctrStd: min ctr
_leakStd: max leak
_ratio: max leak/ctr ratio
_noiseStd: max noise
```

`HBDTEST_ROMAStd` 结构体为相关阈值，可在 `gh3011_example_reg_array.c` 中设置。

```
HBDTEST_ROMAConfigParam: _ledResisLst:the resistance in config
_ledCurrLst:the current in config
```

`HBDTEST_ROMAConfigParam` 结构体为相关配置参数。

`gh3011_example_port.c` 中的 `handle_system_test_result()` 函数为结果处理函数，参数 `test_res` 为测试结果，具体定义为：

- `<0=> ok` 测试成功
- `<1=> order err` 操作错误

- <2=> comm read err 通信读操作错误
- <3=> comm write err 通信写操作错误
- <4=> otp read err OTP 操作错误
- <5=> ctr not pass CTR 测试不通过
- <6=> rawdata not pass Rawdata 测试不通过
- <7=> noise not pass 噪声测试不通过
- <8=> leak not pass 漏光测试不通过
- <9=> leakratio not pass Leak/CTR 测试不通过
- <10=> resource error 源错误

参数 `led_num` 只在出现测试失败时有意义，对应测试不通过的 LED，测试通过则第二个参数始终为 0，无意义。

进行光路灵敏度测试和漏光测试之前，可以设置停顿时间用来放置测试头，停顿时间可在 `gh3011_example_port.c` 中的 `handle_before_system_os_test()` 函数进行设置，该函数需在光路灵敏度测试和漏光测试开始之前被调用。

若测试模式为 `HBDTEST_TEST_ALL_TEST`，LED 0-2 全部置 1 进行测试时，将按照“通信测试→OTP 测试→LED0 光路灵敏度测试、LED1 光路灵敏度测试、LED2 光路灵敏度测试→LED0 漏光测试、LED1 漏光测试、LED2 漏光测试”的顺序依次进行，其余模式和 LED 选择以此类推。光路灵敏度测试和漏光测试将分别测试 100 次，测试次数可在 `gh3011_example_config.c` 中 `__SYSTEM_TEST_DATA_CNT_CONFIG__` 设置，且必须小于等于 100。

2.3.3.2 UART 测试

工程模式下的 MT 测试，即使用 FPM02 工具板进行 UART 测试。该工具板的 UART 接口定义如下所示：

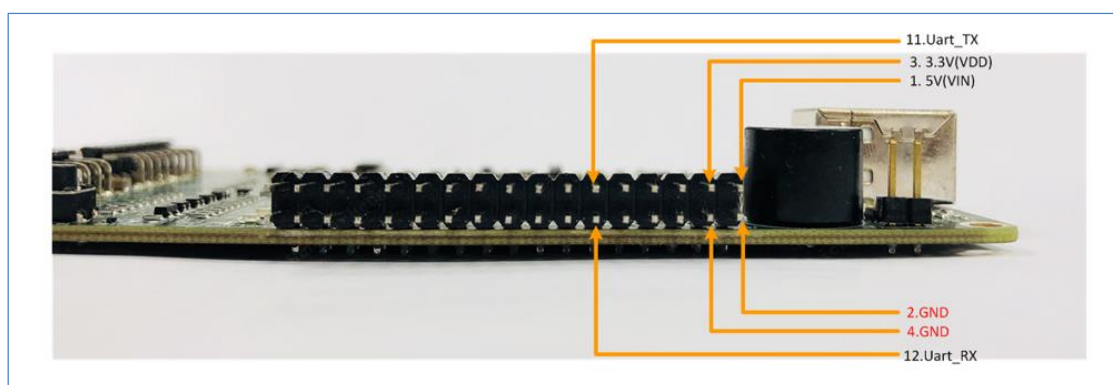


图 2-15 UART 通信接口(J6)定义（侧视图）

各管脚的具体描述，如表 2-4 所示：

表 2-4 UART 通信接口(J6)管脚描述

管脚序号	管脚名称	描述
1	5V (VIN)	5 V 供电电源，根据测试设备需求选择使用
2	GND	连接模组的 GND 网络
3	3.3V (VDD)	3.3 V 供电电源，根据测试设备需求选择使用
4	GND	连接模组的 GND 网络
11	I2C1_SDA	复用 UART Tx，连接模组 UART_TX 网络

12	I2C1_SCL	复用 UART_Rx，连接模组 UART_RX 网络
----	----------	----------------------------

当使用 UART 测试时，需根据实际应用需求，实现 *gh30x_example_port.c* 文件中的 `uart_module_recv_data()` 函数和 `uart_module_send_data()` 函数。除这两个函数以外，还需实现 UART 功能，包括管脚设置、UART 功能设置等。

`uart_module_recv_data` 函数主要用于从 UART 接收控制命令用于返回数据及控制 GH30x 功能；`uart_module_send_data` 函数主要用于发送数据给外部，以便于测试分析。

3 SDK API 说明

GH30x SDK 提供了丰富的 API，可供用户应用层调用。

3.1 驱动层 API

各 API 函数的具体描述，参见下列表格。

函数原型	int gh30x_module_init(void)
功能描述	模组初始化函数，主要包含通信接口（I2C\SPI）的初始化、HOOK 函数注册、算法版本号打印、GH30x 芯片初始化、Gsensor 驱动及 Gsensor 初始化、相关功能变量初始化和相关算法初始化，在修改前需要仔细了解函数功能；
输入参数	无
输出参数	无
返回值	GH30X_EXAMPLE_OK_VAL：成功 GH30X_EXAMPLE_ERR_VAL：失败

函数原型	void gh30x_module_start(GU32 unFunctionMode)
功能描述	模组开启函数
输入参数	unFunctionMode :功能组合： 举例 1: GH30X_FUNCTION_HR GH30X_FUNCTION_HRV、 举例 2: GH30X_FUNCTION_ADT GH30X_FUNCTION_SPO2
输出参数	无
返回值	无

函数原型	void gh30x_module_stop(void)
功能描述	模组停止函数
输入参数	无
输出参数	无
返回值	无

函数原型	void Gh30xDemoFuncEnable (GU32 unFuncMode)
功能描述	功能使能函数
输入参数	unFuncMode :需要使能的功能组合 举例 1: GH30X_FUNCTION_HR GH30X_FUNCTION_HRV、 举例 2: GH30X_FUNCTION_ADT GH30X_FUNCTION_SPO2
输出参数	无
返回值	无

函数原型	void Gh30xDemoFuncDisable(GU32 unFuncMode)
功能描述	功能禁用函数
输入参数	unFuncMode :需要禁用的功能组合 举例 1: GH30X_FUNCTION_HR GH30X_FUNCTION_HRV、 举例 2: GH30X_FUNCTION_ADT GH30X_FUNCTION_SPO2
输出参数	无

返回值	无
函数原型	<pre>void Gh30xDemoSampleStart(const ST_REGISTER* pstBaseArray, GU16 usBaseArrayLen, const ST_REGISTER_BIT_FIELD* pstAppendArray, GU16 usAppendArrayLen, stSampleResetCfg* pstSampleResetCfg)</pre>
功能描述	采样开启函数 使用注意事项：当 Gh30xDemoFuncEnable、Gh30xDemoFuncDisable、Gh30xDemoSampleStop、Gh30xDemoSampleStart 配合使用时，请保证 Gh30xDemoSampleStart 或 Gh30xDemoSampleStop 是最后被调用的；
输入参数	pstBaseArray：基础配置表指针，通常是 hb_adt_confirm_reg_config、hb_reg_config_array、spo2_reg_config_array 中的某一张； usBaseArrayLen：基础配置表长度 pstAppendArray：追加配置表的指针 usAppendArrayLen：追加配置表的长度 pstSampleResetCfg：采样参数重设置（可设置增益、fifo 水线、采样率等参数） pstSampleResetCfg 举例： <pre>const stSampleResetCfg g_stSampleResetCfg = { .usResetEn = 0 RESET_CFG_SAMPLE_RATE_EN //需要重新设定采样率 RESET_CFG_INT_TIME_EN //需要重新设定 ADC 积分时间 RESET_CFG_CHNL0_AGC_FORCE_DIS // 需要禁用通道 0 调光 RESET_CFG_CHNL1_AGC_FORCE_DIS // 需要禁用通道 1 调光 RESET_CFG_CHNL0_DEFAULT_CUREET_EN //需要重新设定通道 0 电流 RESET_CFG_CHNL1_DEFAULT_CUREET_EN //需要重新设定通道 1 电流 RESET_CFG_CHNL0_TIA_GAIN_EN //需要重新设定通道 0 增益 RESET_CFG_CHNL1_TIA_GAIN_EN //需要重新设定通道 1 增益 RESET_CFG_FIFO_WATER_LINE_EN //需要重新设定 fifo 水线 , .usResetSampleRate = 25, .uchResetIntTimeIndex = 0, .uchResetDrvCurrentChnl0 = 0, .uchResetDrvCurrentChnl1 = 0, .uchResetTiaGainChnl0 = 0, .uchResetTiaGainChnl1 = 0, .uchResetFifoWaterLine = 100, };</pre>
输出参数	无
返回值	无

函数原型	void Gh30xDemoSampleStop(void)
功能描述	采样停止函数 使用注意事项：当 Gh30xDemoFuncEnable、Gh30xDemoFuncDisable、Gh30xDemoSampleStop、Gh30xDemoSampleStart 配合使用时，请保证 Gh30xDemoSampleStart 或 Gh30xDemoSampleStop 是最后被调用的；
输入参数	无
输出参数	无
返回值	无

函数原型	GS8 HBD_ChipReset(void)
功能描述	复位芯片
输入参数	无
输出参数	无
返回值	HBD_RET_OK 成功 HBD_RET_COMM_NOT_REGISTERED_ERROR 未注册芯片通信接口

函数原型	GS8 * HBD_GetHbdVersion(void)
功能描述	获取 SDK 版本号字符串
输入参数	无
输出参数	无
返回值	版本号字符串

函数原型	void gh3x2x_get_rawdata_hook_func(GU32 *read_buffer_ptr, GU16 length)
功能描述	获取原始数据 hook
输入参数	read_buffer_ptr : buffer 指针 Bit[30]: 调光标记 Bit[29:27]: 环境光等级 Bit[26:24]: 增益档位值 Bit[23:7]: Rawdata 其他位: 无需关心 length : 数据长度（采样点数）
输出参数	无
返回值	无

函数原型	void gh3x2x_algorithm_get_io_data_hook_func(const STGh30xFrameInfo * const pstFrameInfo)
功能描述	获取功能帧 hook
输入参数	pstFrameInfo 帧数据指针
输出参数	无
返回值	无

函数原型	<code>void handle_wear_status_result(uint8_t wearing_state_val, GU8 uchLivingFlag)</code>
功能描述	佩戴结果报告函数
输入参数	wearing_state_val : WEAR_STATUS_WEAR 佩戴 WEAR_STATUS_UNWEAR 脱落 uchLivingFlag: 佩戴或脱落属性 0: 物体 1: 活体 举例一: wearing_state_val = WEAR_STATUS_WEAR, uchLivingFlag =0 : 对物体佩戴 举例二: wearing_state_val = WEAR_STATUS_WEAR, uchLivingFlag =1 : 活体佩戴 举例三: wearing_state_val = WEAR_STATUS_UNWEAR, uchLivingFlag =0 : 对空脱落 举例四: wearing_state_val = WEAR_STATUS_UNWEAR, uchLivingFlag =1 : 非活体脱落
输出参数	无
返回值	无

3.2 算法调用层 API

3.2.1 函数描述

函数原型	<code>void GH30X_AdtAlgorithmResultReport(STGh30xAlgoResult * pstAlgoResult, GU32 lubFrameId)</code> <code>void GH30X_HrAlgorithmResultReport(STGh30xAlgoResult * pstAlgoResult, GU32 lubFrameId)</code> <code>void GH30X_Spo2AlgorithmResultReport(STGh30xAlgoResult * pstAlgoResult, GU32 lubFrameId)</code> <code>void GH30X_HrvAlgorithmResultReport(STGh30xAlgoResult * pstAlgoResult, GU32 lubFrameId)</code> <code>void GH30X_SoftAdtAlgorithmResultReport(STGh30xAlgoResult * pstAlgoResult, GU32 lubFrameId)</code>
功能描述	各算法计算结果的报告函数
输入参数	pstAlgoResult : <pre>typedef struct { GU8 uchUpdateFlag;//更新标记 GU8 uchResultNum;//结果数量 GU16 usResultBit;//结果更新 bit GS32 snResult[];//结果 buffer，定义见结果数组描述章节 } STGh3x2xAlgoResult;</pre> 结果更新 bit 指每个 bit 相当于结果更新的 Flag，比如 0x3 指的是 buffer 中第 0 和第 1 个结果更新了，其它未更新
输出参数	无
返回值	无

3.2.2 算法结果数组描述

3.2.2.1 HR

Result_Index	描述	备注
snResult[0]	心率值（单位：bpm）	无
snResult[1]	置信度分数（0 ~ 100）	只有 S 版本支持
snResult[2]	信噪比：滤波后频谱主峰能量与总能量之比	暂未启用

snResult[3]	置信度等级，目前输出 2 档置信度，1 表示可信，0 表示不可信	只有 S 版本支持
snResult[4]	Acc_info	暂未启用
snResult[5]	Acc_scence	暂未启用

3.2.2.2 SPO2

Result_Index	描述	备注
snResult[0]	相对血氧饱和度（单位：%）	无
snResult[1]	综合 R 值*10000	无
snResult[2]	置信度：各路通道输出的实时置信度（即出值策略处理后的），范围 0~100；	无
snResult[3]	置信等级：范围-2~5，值越大，输出值依赖策略卡控的成分越少，负值表示异常 -1：表示当前秒的实时计算结果未被推进缓存队列或缓存队列长度不足，可能出现在首次出值前若干秒或当前秒存在运动调光等异常； -2：之前出过值，但长时间信号异常，缓存队列长度不足，重置状态机为初始态； -3：当前信号较差	无
snResult[4]	心率值（暂无用）	暂未启用
snResult[5]	数据无效标记，无效时 bit 位为 1 bit0:运动异常标记 bit1:手表朝向异常标记 bit2:调光标记 bit3:R 值无效标记	暂未启用

3.2.2.3 HRV

Result_Index	描述	备注
snResult[0]	RRI0 值（单位：ms）	无
snResult[1]	RRI1 值（单位：ms）	无
snResult[2]	RRI2 值（单位：ms）	无
snResult[3]	RRI3 值（单位：ms）	无
snResult[4]	置信度：0 不可信，25 低置信度，75 高置信度，100 可信	无
snResult[5]	RRI 数量	无

备注：HRV 结果每秒输出一次，每次输出的为最新 1s 内的 RRI 个数，以及各个 RRI 的时间值

3.2.2.4 NADT

Result_Index	描述
snResult[0]	佩戴结果： 0：尚未确定 1：佩戴

	2: 脱落
snResult[1]	佩戴结果属性: 0: 物体 (对物体佩戴或对空脱落) 1: 活体 (活体佩戴或非活体脱落) 举例一: snResult[0]=1, snResult[1]=0 : 对物体佩戴 举例二: snResult[0]=1, snResult[1]=1 : 活体佩戴 举例三: snResult[0]=2, snResult[1]=0 : 对空脱落 举例四: snResult[0]=2, snResult[1]=1 : 非活体脱落

4 常见问题及注意事项

1. 更换寄存器数组在哪个文件？对应模式的寄存器数组是哪些？

gh30x_example_reg_array.c 文件；

- hb_adt_confirm_reg_config: 心率 ADT 检测寄存器数组
- hb_reg_config_array: 心率检测寄存器数组
- spo2_reg_config_array: 血氧检测寄存器数组

需要进行寄存器数组更改时，目前只需关注以上数组的更改，常见的修改频率的寄存器地址为：0x0016；对应的频率为：0x51e(25HZ)、0x0147(100HZ)。

2. 算法对于 G-sensor 的要求？

G-sensor 的传入数量一定需要大于 Rawdata 的数量，否则，心率计算接口等会吐出 0 值；确保 G-sensor 的频率大于或等于心率 IC 的采样率，并确保时间轴上对齐。具体可参考 [2.3.1 G-sensor 调试](#)。

3. 怎么确认通信正常？

不包含 GoodixSDK，可按以下步骤操作：

(1) 实现 IIC/SPI 接口函数。

(2) IIC 地址为 0x28（8 bits），按照 IIC 发送命令协议发送 0xDD 0xDD 0xC0 命令，延时 1 ms 后，按照 IIC 读操作协议从地址为 0x0028 的寄存器中读取 2 个字节，若得到值为 0x0031，则通信接口验证成功。

(3) 按照 SPI 发送命令协议发送 0xC0 命令，延时 1 ms 后，按照 SPI 读操作协议从地址为 0x0028 的寄存器中读取 2 个字节，若读取的值为 0x0031，则通信接口验证成功。

包含 GoodixSDK 时，可按以下步骤操作：

(1) 实现 IIC/SPI 接口函数。

(2) 调用 GoodixSDK 的 HBD_SetI2CRW/ HBD_SetSPIRW 函数将 IIC 接口注册到库中。

(3) 调用 GoodixSDK 的 HBD_CommunicationInterfaceConfirm 接口，若返回 HBD_RET_OK，则通信接口验证成功。

4. 对 IIC 速率有什么要求？

最大支持 400 kHz，一般使用 400 kHz。

当使用模拟 IIC 的时候，可能会存在读取 IIC 数据不及时的问题，所以建议使用硬件 IIC。

5. GH30x 的 ADC 参考电压及采样精度是多少？

ADC 参考电压为 1.6V，采样精度是 17bits。

6. GH30x 采样得到的 Rawdata 与实际的电流值的转换公式是什么？

$$IPD = \text{rawdata} / 65536 * 0.8 / \text{gain} * 1000000 \text{ nA} \quad (\text{gain 单位 K})$$

7. 双通道开启时，采集到的双通道数据如何分布？

双通道数据按虚拟通道值从小到大，在 FIFO 中反复依次排列。

8. 最高可以支持到多少采样率？

GH30x 最高可以支持到单通道 1K 采样率，但 1K 采样率无法支持 HBD_L 模式，功耗会增加。

9. 移植出现算法库或驱动库编译报错，怎么排查问题？

首先需要判断是哪个库编译报错，先把算法功能的宏全部关闭，只链接驱动库，如果能编译通过说明驱动库没有问题；然后依次放开算法功能的宏，判断是哪个算法库无法链接，根据不同平台的链接优化特性，算法库的链接顺序推荐是把 common 库放到最后链接；

 说明：

SDK 的所有接口调用，推荐采用单任务方式，心率的所有接口调用都在一个任务中执行，中断也建议通过消息发给单线程任务来执行中断函数体；对于多任务的调用，需对部分接口加互斥锁。

5 附录

5.1 心率场景定义

心率模式可以设置场景以提高准确性，每次在 `gh30x_module_start` 前调用 `GH30X_HBA_SCENARIO_CONFIG` 即可完成设置，参数范围 0~23，具体描述如下：

表 5-1 心率场景参数说明

参数	说明
0	Default（由算法内部识别处理）
1	Routine（日常生活）
2	Indoor running（室内跑步）
3	Indoor walking（室内步行）
4	Stair activity（上下楼梯）
5	Outdoor running（户外跑步）
6	Outdoor walking（户外步行）
7	Tranquillization（静息）
8	Rest（休息）
9	Short of breath（憋气）
10	Indoor cycling（室内自行车）
11	Outdoor cycling（室外自行车）
12	Bicycle motocross（室外自行车越野）
13	High heart rate running（高心率跑步）
14	Combination running of treadmill（跑步机组合跑）
15	High intensity exercise combination（高强度运动组合）
16	Traditional strength training combination（传统力量训练组合）
17	Step-on testing（台阶测试）
18	Ball game（球类运动）
19	Aerobics（健身操）
20	Sleep（睡眠）
21	Jump rope（跳绳）
22	Cordless jump rope（无绳跳绳）
23	Swimming（游泳）